

Lab 3: IEEE 802.15.4

General Instructions

In previous labs, we explored Bluetooth Low Energy (BLE). BLE is intended for short-range, low-power communication and typically uses a star topology, where a central device connects to multiple peripherals — making it well-suited for scenarios like sensor data collection.

In this lab, we turn our attention to 802.15.4, a standard protocol for low-rate wireless personal area networks (LR-WPANs). 802.15.4 is designed for low-power, low-data-rate applications and supports various network topologies, including star, tree, and mesh. You can also use 802.15.4 to transmit IPv6 packets using the 6LoWPAN adaptation layer. This enables communication between 802.15.4 devices and the broader Internet using a gateway device, which is a key aspect of the Internet of Things (IoT).

Thus, we can then use standard Internet protocols, such as UDP or CoAP, to send messages between devices. This makes 802.15.4 a powerful choice for IoT applications that require low power consumption and reliable communication over short distances.

In this lab, you will implement a simple communication scenario using 802.15.4. Two devices will form a connection and exchange (simulated) sensor data and commands using the CoAP protocol. You will evaluate the reliability and timing of your setup.

Part 1 – Connection Setup (3 points)

Form a connection between two 802.15.4 devices. You can use the same hardware as in previous labs, but make sure to configure it to use 802.15.4 instead of BLE. There are some hints in the lab presentation on how to do this. You can also refer to the zephyr documentation, although it is sadly not very detailed on this topic.

Then implement a simple UDP communication between the two devices. One device will act as a sender, sending a datagram with (simulated) sensor data regularly. The other device will act as a receiver, listening for incoming datagrams, parsing the data, and printing it to the console.

Hint: Use static IPv6 addresses for the devices to simplify the setup.

Note: IEEE 802.15.4 emulation in Renode might not work reliably. If you encounter issues, you can test your implementation on real hardware, such as the nRF52840 DKs.

Note: If you are using a BBC micro:bit with west, you might have to enable the IEEE 802.15.4 driver using the `app.overlay` configuration file provided with the task.

Part 2 – CoAP Server (1.5 points)

Using your UDP communication setup, implement a simple CoAP server. The server should support at least two endpoints, one for a sensor reading (e.g., /sensor) and one for an action (e.g., /action). The sensor endpoint should at least support GET requests; the action endpoint should at least support POST or PUT requests. You can use the Zephyr CoAP library to implement the server functionality. The server should parse incoming CoAP requests, perform the appropriate actions (e.g., return a simulated sensor reading or trigger an action), and send back CoAP responses.

You do not need to support encryption or authentication for this lab.

Note: If you are using platformio, the zephyr CoAP server subsystem will not be available. The CoAP library is still available.

Hint: You can debug your implementation for this part using Renode via an emulated Ethernet interface. There will be some instructions in the lab materials soon on how to set this up.

Part 3 – CoAP Client (1.5 points)

Using the same UDP communication setup, implement a simple CoAP client on the other device. The client should periodically send GET requests to the sensor endpoint of the server and print the received sensor readings to the serial console. Additionally, the client should be able to send POST or PUT requests to the action endpoint of the server based on a button press.

Part 4 – Evaluation (2 points)

Evaluate the reliability and timing of your setup. Answer for example these questions:

- How reliable is the communication between the devices? Do you experience any packet loss or delays? If so, under what conditions?
- How does the timing of your CoAP requests and responses look? Are there any noticeable delays? How does the timing change under different conditions (e.g., with more frequent requests, with interference, etc.)?
- How does the performance of your setup compare to BLE communication?
- Does the triggering of the action endpoint work reliably? Does it feel responsive?
- ...